

# TeleFlow

PLATFORM

Business & Software as Code

---

Documentación Técnica Completa

v1.0.0 · Junio 2026 · Confidencial

FastAPI

Lark LALR

PostgreSQL

Redis

RabbitMQ

# Tabla de contenidos

---

|                                                  |    |
|--------------------------------------------------|----|
| Visión y propuesta de valor .....                | 3  |
| Arquitectura del sistema .....                   | 5  |
| TeleFlow DSL v2 — Referencia .....               | 8  |
| Arquitectura event-driven y API .....            | 14 |
| Despliegue y stack tecnológico .....             | 21 |
| Decisiones de arquitectura (ADRs) .....          | 27 |
| Apéndice A — Dominio Ceibal (ceibal.tflow) ..... | 30 |
| Apéndice B — Quick start .....                   | 31 |

## Visión y propuesta de valor

# 1 Visión y propuesta de valor

TeleFlow es una plataforma de orquestación empresarial basada en el principio **Business & Software as Code**. Permite a organismos públicos y privados modelar, desplegar y ejecutar sus procesos de venta y post-venta como código declarativo, con el mismo rigor con que Terraform gestiona infraestructura.

### 1.1 El ciclo de vida de un proceso



### 1.2 Un producto, muchos organismos

TeleFlow **no es SaaS**. Es un producto que se instancia por organismo, como GitLab self-hosted. Cada organismo posee y opera su propia instancia con datos completamente aislados.

El mismo `docker-compose.yml` y los mismos Helm charts sirven a todos. Los templates de proceso (TMForum, ITSM, etc.) se comparten como repositorios Git.

#### i Analogía

TeleFlow es a los procesos de negocio lo que **Terraform** es a la infraestructura: código versionado, revisable y reproducible.

#### Organismos cliente (ejemplos)

|               |                                         |
|---------------|-----------------------------------------|
| OSE           | Empresa pública de agua y saneamiento   |
| Antel         | Operador nacional de telecomunicaciones |
| Ceibal        | Plan educativo 1:1 de Uruguay           |
| Min. Economía | Gestión de subsidios y transferencias   |

### 1.3 Analogías de diseño

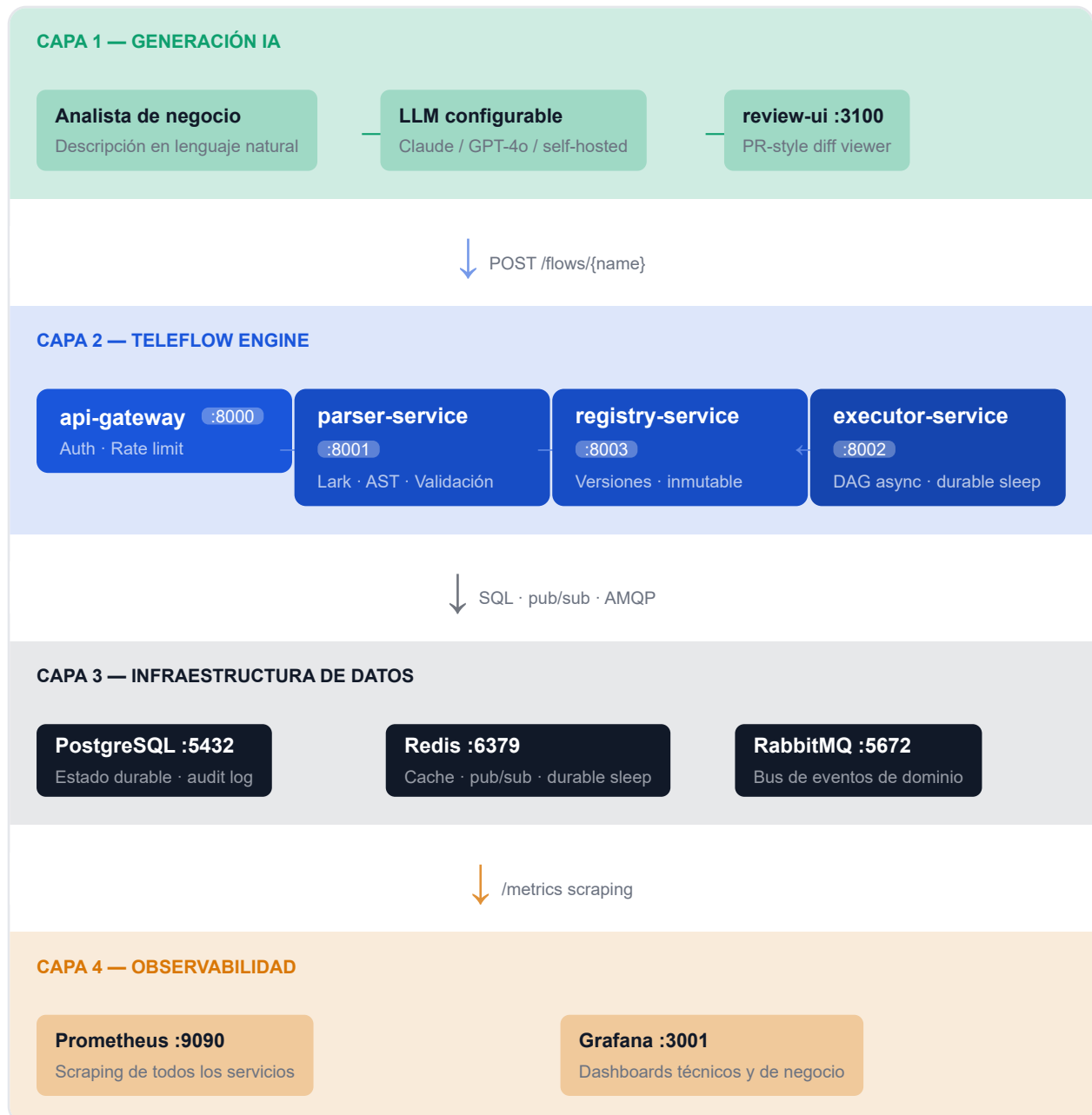
| Concepto TeleFlow | Analogía técnica | Analogía de negocio |
|-------------------|------------------|---------------------|
|-------------------|------------------|---------------------|

|                             |                            |                              |
|-----------------------------|----------------------------|------------------------------|
| Archivo <code>.tflow</code> | Helm chart / K8s manifest  | Especificación del proceso   |
| <code>`entity`</code>       | Aggregate (DDD)            | Niño, cliente, contrato      |
| <code>`process`</code>      | Kubernetes Deployment      | Proceso de venta, soporte    |
| <code>`relation`</code>     | Join con estado propio     | Inscripción, contrato activo |
| Registry Service            | OCI Registry / Helm repo   | Repositorio de procesos      |
| Executor Service            | Kubernetes controller loop | Motor de ejecución           |
| Instance                    | Pod en ejecución           | Venta en curso               |
| State Store                 | etcd del cluster           | Estado durable del negocio   |

# Arquitectura del sistema

## 2 Arquitectura general

### 2.1 Capas del sistema



## 3 Arquitectura de microservicios

## 3.1 Responsabilidades por servicio

| Servicio         | Puerto | Responsabilidad única                                                                                                                                                               |
|------------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| api-gateway      | 8000   | Único punto de entrada externo. Auth (X-TeleFlow-API-Key), token-bucket rate limiting por key, routing transparente a servicios internos.                                           |
| parser-service   | 8001   | Recibe source .tflow y produce AST validado en JSON. Valida sintaxis (Lark LALR) Y semántica (estados, transiciones, referencias). Nunca ejecuta nada.                              |
| registry-service | 8003   | Almacena y versiona definiciones. Una versión registrada es inmutable (409 si se intenta reemplazar). Expone `latest` como pointer mutable.                                         |
| executor-service | 8002   | Construye el DAG del flow con networkx, recorre steps async. Durable sleep para human_task. Rule engine (on_event/on_state/on_timer/on_relation). Vista 360. Entity/Relation state. |
| composer-service | 8004   | Abstracción LLM. Genera borradores .tflow desde lenguaje natural. Guarda drafts en Postgres para revisión PR-style.                                                                 |
| review-ui        | 3100   | UI React+Vite para revisión PR-style de flows generados por IA. Diff viewer LCS personalizado.                                                                                      |

## 3.2 Flujo de deploy de un flow

- **POST /flows/{name}**  
Source .tflow + version + descripcion. Header: X-TeleFlow-API-Key
- **Valida sintaxis y semantica**  
Lark LALR parsea. Transformer genera AST. Validator chequea coherencia.
- **Persiste version inmutable**  
Guarda source, AST JSON y checksum SHA-256. 409 si version ya existe.
- **Respuesta inmediata**  
{ flow\_id, name, version, status: registered }

## 3.3 Flujo de ejecución de proceso

Solicitud (POST /execute)

json · Request

```
POST /execute
{
  "flow_name": "venta_internet_hogar",
```

Consulta de estado (GET /instances/{id})

json · Response

```
{
  "id": "uuid-...",
  "status": "WAITING_SIGNAL",
```

```
"version": "latest",  
"payload": {  
  "cliente_id": "uuid-...",  
  "producto": "internet_100"  
},  
"correlation_id": "ext-uuid-123"  
}
```

```
← { "instance_id": "uuid",  
    "status": "TRIGGERED" }
```

```
"flow_name": "venta_internet_hogar",  
"current_step": "aprobacion_gerencia",  
"context": {  
  "_cursor": {  
    "stage": 2,  
    "step": 0  
  }  
}
```

## TeleFlow DSL v2 — Referencia

# 4 TeleFlow DSL v2 — Referencia

El DSL es el corazón del producto. Un archivo `.tflow` es una declaración completa del dominio de negocio: entidades, relaciones, procesos, reglas e integraciones. La gramática EBNF vive en `teleflow/dsl/teleflow.lark` y es la fuente de verdad del lenguaje (ADR-001).

### 4.1 Bloques disponibles

| Bloque                     | Estado    | Descripción                                                                                                                                     |
|----------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>`entity`</code>      | NUEVO v2  | Objeto de negocio con identidad, campos tipados, máquina de estados explícita, invariantes y eventos emitidos por transición.                   |
| <code>`relation`</code>    | NUEVO v2  | Vínculo entre dos entities con su propio estado y ciclo de vida. El «producto instanciado» (ej: inscripción niño↔curso).                        |
| <code>`rule`</code>        | NUEVO v2  | Regla de negocio event-driven. Cuatro tipos: <code>on_event</code> , <code>on_state</code> , <code>on_timer</code> , <code>on_relation</code> . |
| <code>`view360`</code>     | NUEVO v2  | Declara la vista 360 de un entity: estado, relaciones activas, procesos en vuelo, historial de eventos, alertas.                                |
| <code>`process`</code>     | existente | Orquesta steps en stages (sequential, parallel, decision). Puede emitir eventos y modificar estado de entities.                                 |
| <code>`step`</code>        | existente | Tarea atómica reutilizable: <code>automated</code> , <code>human_task</code> , <code>decision</code> , <code>notification</code> .              |
| <code>`integration`</code> | existente | Conectores externos: REST, AMQP/Kafka, SMTP, SMS. Credenciales via <code>`\${env.VAR}`</code> .                                                 |
| <code>`catalog`</code>     | existente | Productos ofertables. Puede referenciar una entity como su instancia al contratarse.                                                            |
| <code>`party`</code>       | existente | Actores del proceso. Puede vincularse a una entity.                                                                                             |

### 4.2 Bloque entity

#### Sintaxis completa

```
tflow · entity

entity "nino" {
  description: "Estudiante Ceibal"

  fields {
    ci:          string  required unique
    nombre:     string  required
    fecha_nac:  date     required
  }
}
```

#### Tipos de campo

| Tipo                    | Descripción        |
|-------------------------|--------------------|
| <code>`string`</code>   | Texto libre        |
| <code>`number`</code>   | Entero o decimal   |
| <code>`date`</code>     | Fecha (YYYY-MM-DD) |
| <code>`datetime`</code> | Fecha+hora con TZ  |
| <code>`bool`</code>     | true / false       |



```

departamento: string
nivel:          enum["primaria","secundaria","utu"]
dispositivo: string optional
}

lifecycle {
  initial: "REGISTRADO"
  states ["REGISTRADO","ACTIVO","INSCRIPTO",
          "GRADUADO","INACTIVO"]

  transitions {
    REGISTRADO -> ACTIVO      via "activar"
    ACTIVO      -> INSCRIPTO  via "inscribir"
    INSCRIPTO   -> ACTIVO      via "desinscribir"
    ACTIVO      -> GRADUADO   via "graduar"
    ACTIVO      -> INACTIVO   via "desactivar"
    INACTIVO    -> ACTIVO      via "reactivar"
  }
}

events {
  on_transition "activar"      emit "nino.activado"
  on_transition "inscribir"    emit "nino.inscripto"
  on_transition "graduar"      emit "nino.graduado"
  on_field_change "nivel"      emit
    "nino.nivel_cambiado"
}

invariants {
  "edad >= 5 AND edad <= 18"
  "nivel != null WHEN estado == INSCRIPTO"
}
}

```

``enum[...]`` Conjunto fijo de valores

### Modificadores de campo

| Modificador                    | Efecto              |
|--------------------------------|---------------------|
| <code>`required`</code>        | No nulo en create   |
| <code>`optional`</code>        | Puede ser null      |
| <code>`unique`</code>          | Índice único en DB  |
| <code>`default(v)`</code>      | Valor por defecto   |
| <code>`range(min, max)`</code> | Validación de rango |

#### i Invariantes

Se evalúan en cada PATCH y en cada transición de estado. Sintaxis: expresión booleana del evaluador TeleFlow.

`WHEN cond` hace que el invariante solo aplique cuando la condición es verdadera.

## 4.3 Bloque relation

```

tflow · relation

relation «inscripcion» { description: «Niño ↔ Curso»
from:  entity.nino  to:  entity.curso  cardinality:
«many_to_many» constraint: «max 3 cursos simultá-
neos»

lifecycle { initial: «PENDIENTE» states [«PENDIEN-
TE»,«ACTIVA», «COMPLETADA»,«ABANDONADA»]
transitions { PENDIENTE -> ACTIVA via «activar» AC-
TIVA -> COMPLETADA via «completar» ACTIVA ->
ABANDONADA via «abandonar» }}

fields { fecha_inscripcion: date required modalidad:
enum[«presencial»,«virtual»] progreso: number de-
fault(0) range(0,100) ultimo_acceso: datetime optional }

events { on_transition «activar» emit
«inscripcion.activada» on_transition «completar» emit
«inscripcion.completada» on_field_change «progreso»
emit «inscripcion.progreso_actualizado» }}

```

### Cardinalidades soportadas

| Valor                       | Semántica       |
|-----------------------------|-----------------|
| <code>"one_to_one"</code>   | 1 : 1           |
| <code>"one_to_many"</code>  | 1 : N           |
| <code>"many_to_many"</code> | N : M (default) |

#### ✓ API de relaciones

- `POST /relations/{type}` — crear relación
- `POST /relations/{type}/{id}/transition` — transicionar
- `PATCH /relations/{type}/{id}` — actualizar campos

#### i Identidad

La relación se identifica por

(relation\_type, from\_id, to\_id) . Los eventos incluyen from\_id y to\_id para que las reglas puedan extraerlos con event.from\_id / event.to\_id .

## 4.4 Bloque rule

### on\_event — inmediato

```
rule "activar_acceso_al_inscribirse" {
  on_event: "inscripcion.activada"
  execute: process.activacion_acceso_plataforma
  with {
    nino_id: event.from_id
    curso_id: event.to_id
  }
}
```

### on\_event + condición

```
rule "prioridad_dispositivo" {
  on_event: "nino.activado"
  condition: event.entity.dispositivo == null
  execute: process.asignacion_dispositivo
  with {
    nino_id: event.entity_id
  }
}
```

### on\_timer — temporal

```
rule "detectar_abandono" {
  on_timer {
    after: 30 days
    since: "inscripcion.activada"
    condition: days_since(
      relation.inscripcion.ultimo_acceso
    ) >= 30 OR
    relation.inscripcion.ultimo_acceso == null
  }
  execute: process.reenganche_estudiante
  with {
    nino_id: relation.from_id
    curso_id: relation.to_id
  }
}
```

### Tipos de trigger

| Tipo          | Cuándo dispara                                 |
|---------------|------------------------------------------------|
| `on_event`    | Al recibir el evento del bus RabbitMQ          |
| `on_state`    | Cuando entity/relation alcanza un estado       |
| `on_timer`    | Si condición persiste N días después de evento |
| `on_relation` | Al crear/modificar/eliminar una relation       |

## 4.5 Bloque process y stages

### Stage sequential

```
stage "alta" {
  mode: sequential
  steps [
    step.crear_credenciales_lms,
    step.notificar_acceso
  ]
}
```

### Stage decision (con branches)

```
stage "evaluacion_credito" {
  mode: decision
  steps [step.consultar_buro]
  when signals.aprobacion_gerencia.signal
    == "approve" -> stage.activacion
  else -> stage.rechazo
}
```

## Stage parallel

```
stage "contacto" {
  mode: parallel
  steps [
    step.notificar_tutor_abandono,
    step.recordatorio_estudiante
  ]
}
```

## Modos de stage

| Modo         | Comportamiento                                                                |
|--------------|-------------------------------------------------------------------------------|
| `sequential` | Steps en serie. El cursor avanza paso a paso.                                 |
| `parallel`   | Steps con <code>asyncio.gather()</code> . No admite <code>human_task</code> . |
| `decision`   | Evalúa branches. Salta a otro stage o a <code>stage.end</code> .              |

## 4.6 Bloque step — atributos

| Atributo      | Tipo     | Aplica a       | Descripción                                                   |
|---------------|----------|----------------|---------------------------------------------------------------|
| `type`        | enum     | todos          | automated   human_task   decision   notification              |
| `integration` | ref      | automated      | Referencia al bloque integration a usar                       |
| `method`      | string   | automated REST | Verbo HTTP: GET, POST, PUT, PATCH, DELETE                     |
| `path`        | string   | automated REST | Path del endpoint. Soporta interpolación <code>\${...}</code> |
| `payload`     | block    | automated      | Pares clave: expresión para el body de la llamada             |
| `retries`     | number   | automated      | Intentos en caso de error (default: 0)                        |
| `timeout`     | duration | automated      | Ej: 30 seconds, 5 minutes                                     |
| `assignee`    | string   | human_task     | Rol o usuario que debe resolver la tarea                      |
| `signals`     | list     | human_task     | Valores válidos de signal aceptados                           |
| `channel`     | string   | notification   | email   sms   amqp                                            |
| `to`          | expr     | notification   | Destinatario (puede ser <code>payload.field</code> )          |
| `template`    | string   | notification   | Mensaje con interpolación <code>{payload.field}</code>        |
| `condition`   | expr     | decision       | Condición booleana. False → branch else                       |

## 4.7 Evaluador de expresiones

### Operadores

| Operador                              | Ejemplo                                 |
|---------------------------------------|-----------------------------------------|
| <code>==, !=</code>                   | <code>`estado == ACTIVO`</code>         |
| <code>&gt;=, &lt;=, &gt;, &lt;</code> | <code>`progreso &gt;= 80`</code>        |
| <code>AND, OR, NOT</code>             | <code>`x &gt;= 5 AND x &lt;= 18`</code> |
| <code>`in`</code>                     | <code>`estado in ["A","B"]`</code>      |
| <code>`WHEN cond`</code>              | <code>`expr WHEN estado == X`</code>    |

### Contexto disponible en expresiones

| Prefijo                       | Apunta a                               |
|-------------------------------|----------------------------------------|
| <code>`event.*`</code>        | Campos del evento RabbitMQ actual      |
| <code>`event.entity.*`</code> | Estado del entity que emitió el evento |
| <code>`relation.*`</code>     | Campos de la relation sujeto           |

**Funciones built-in**

| Función                           | Resultado        |
|-----------------------------------|------------------|
| <code>`days_since(field)`</code>  | Días desde fecha |
| <code>`years_since(field)`</code> | Años desde fecha |

|                                       |                                                           |
|---------------------------------------|-----------------------------------------------------------|
| <code>`payload.*`</code>              | Payload del proceso en ejecución                          |
| <code>`signals.&lt;step&gt;.*`</code> | Data de la señal recibida en <code>human_task</code>      |
| <code>`instance.payload.*`</code>     | Payload de la instancia (en <code>view360 filter</code> ) |
| <code>nombre bare (sin punto)</code>  | Literal string — para comparar con estados                |

**i Null safety**

Todas las comparaciones son null-safe.

`x >= 30 OR x == null` es válido: si `x` es null, la parte `x >= 30` devuelve false y la condición completa se evalúa correctamente.

## 5 Ciclos de vida

### 5.1 Máquina de estados — entity nino (Ceibal)



### 5.2 Estados de una instancia de proceso



**FAILED**

Step agota reintentos

**RETRYING**

POST /retry → retoma desde step fallido

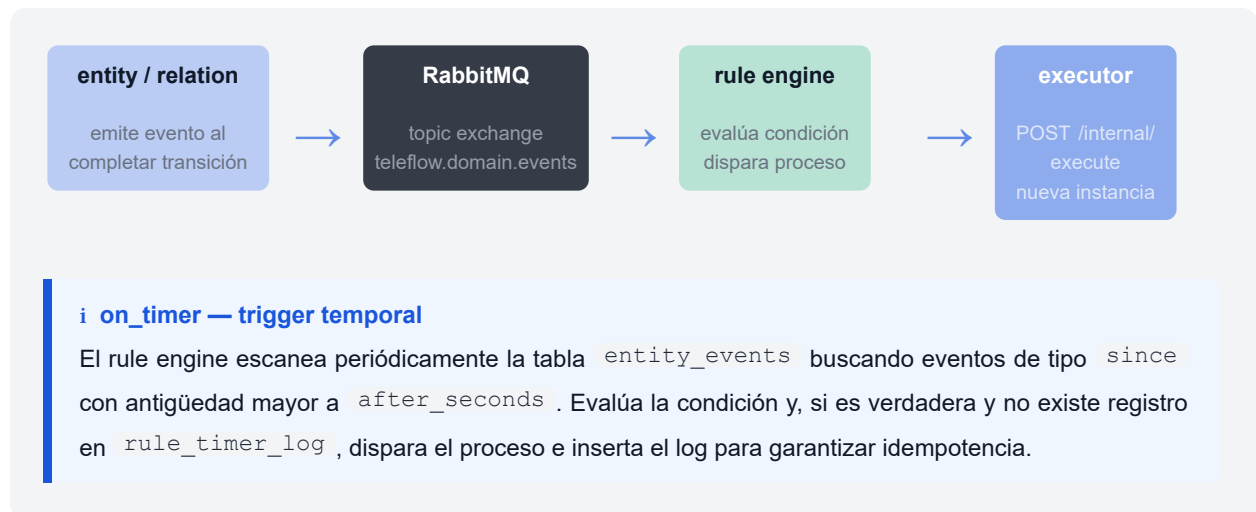
**COMPENSATED**

Rollback exitoso

## Arquitectura event-driven y API

# 6 Arquitectura event-driven

## 6.1 Modelo de tres capas



## 6.2 Flujo completo del dominio Ceibal

|   |                                     |                                          |                                         |
|---|-------------------------------------|------------------------------------------|-----------------------------------------|
| 1 | EVENTO<br>curso.disponible          | REGLA<br>notificar_oferta                | PROCESO<br>notificacion_oferta_curso    |
| 2 | EVENTO<br>inscripcion.activada      | REGLA<br>activar_acceso_al_inscribirse   | PROCESO<br>activacion_acceso_plataforma |
| 3 | EVENTO<br>nino.activado (sin disp.) | REGLA<br>prioridad_dispositivo           | PROCESO<br>asignacion_dispositivo       |
| 4 | EVENTO<br>inscripcion.completada    | REGLA<br>emitir_certificado_al_completar | PROCESO<br>emision_certificado          |
| 5 | EVENTO<br>timer: 30 días sin acceso | REGLA<br>detectar_abandono               | PROCESO<br>reenganche_estudiante        |

# 7 Vista 360 del objeto de negocio

## 7.1 Arquitectura de la vista 360

El endpoint `GET /entities/{type}/{id}/360` agrega en paralelo cinco fuentes de datos y devuelve una vista única del objeto de negocio.

| Fuente            | Descripción                             |
|-------------------|-----------------------------------------|
| entity_state      | Estado actual y campos del entity       |
| relation_state    | Relaciones activas e históricas         |
| process_instances | Procesos en vuelo filtrados por payload |
| entity_events     | Línea de tiempo de eventos              |
| rule conditions   | Alertas evaluadas en tiempo real        |

### Ejemplo de respuesta

```
{
  "entity": "nino",
  "id": "uuid-juan-perez",
  "estado_actual": "INSCRIPTO",
  "campos": {
    "nombre": "Juan Pérez",
    "nivel": "secundaria",
    "departamento": "Montevideo",
    "dispositivo": "X0-4 asignado"
  },
  "relaciones_activas": [{
    "tipo": "inscripcion",
    "curso": "Robotica Básica",
    "estado": "ACTIVA",
    "progreso": 65,
    "ultimo_acceso": "2026-06-10"
  }],
  "procesos_activos": [{
    "flow": "asignacion_dispositivo",
    "status": "IN_PROGRESS",
    "current_step": "verificar_stock"
  }],
  "alertas": [{
    "rule": "detectar_abandono",
    "severidad": "warning",
    "mensaje": "18 días sin acceso"
  }],
  "timeline": [
    { "evento": "nino.inscripto", "fecha": "2026-06-08",
      { "evento": "nino.activado",
        "fecha": "2026-05-15" },
    { "evento": "nino.registrado", "fecha": "2026-05-15" }
  ]
}
```

## 8 Durable sleep — procesos de larga duración

### 8.1 Flujo de durable sleep

executor-service

#### Llega a step human\_task

Persiste instancia → WAITING\_SIGNAL en Postgres con cursor completo

executor-service

#### Suscribe a Redis

SUBSCRIBE teleflow:sigal:{instance\_id} — worker liberado, sin polling

*días o semanas después...***api-gateway****POST /instances/{id}/signal**`{ signal: "approve", actor_id: "juan.p", signal_data: {...} }`**executor-service****Mensaje Redis recibido**

Instancia → IN\_PROGRESS, continúa DAG desde step siguiente

## 8.2 Garantías del durable sleep

**Sin pérdida en restart**

El executor puede reiniciarse en cualquier momento. Al arrancar recupera todas las instancias IN\_PROGRESS y WAITING\_SIGNAL desde Postgres.

**Latencia de ms**

La reactivación llega por Redis pub/sub. No hay timers ni polling que introduzcan latencia artificial.

**Idempotencia**

La señal tiene signal\_key único. Múltiples envíos del mismo signal\_id no crean efectos duplicados (IntegrityError → silencioso).

**Audit log completo**

Cada señal registra quién la envió (actor\_id), cuándo, y con qué datos. La tabla signals es el log de aprobaciones.

## 9 Referencia de la API

**i Autenticación**

Todos los endpoints requieren el header `X-TeleFlow-API-Key: <key>`. La key se configura en la variable de entorno `TELEFLOW_API_KEY`. Las peticiones sin key o con key inválida reciben `401 Unauthorized`.

### 9.1 Flows

**POST****/flows/{name}****API-KEY**

Registra o actualiza un flow (parse + deploy)

**REQUEST BODY**

```
{ "source": "<contenido .tflow>", "version": "1.0.0", "description": "..."
```

**RESPONSE**

```
{ "flow_id": "uuid", "name": "...", "version": "1.0.0", "status": "registered" }
```



**GET** `/flows` **API-KEY** Lista todos los flows registrados

**RESPONSE**

```
[{ "name": "...", "version": "...", "latest": true }]
```

**GET** `/flows/{name}` **API-KEY** Obtiene metadata y source de la versión latest

**RESPONSE**

```
{ "name": "...", "version": "...", "source": "...", "ast": {...} }
```

**GET** `/flows/{name}/versions` **API-KEY** Lista todas las versiones del flow

**RESPONSE**

```
[{ "version": "1.0.0", "created_at": "..."}]
```

## 9.2 Ejecución de procesos

**POST** `/execute` **API-KEY** Dispara una nueva instancia de proceso (respuesta inmediata)

**REQUEST BODY**

```
{ "flow_name": "...", "version": "latest", "payload": {...}, "correlation_id": "..." }
```

**RESPONSE**

```
{ "instance_id": "uuid", "status": "TRIGGERED" }
```

**GET** `/instances/{id}` **API-KEY** Estado actual de una instancia

**RESPONSE**

```
{ "id": "uuid", "status": "IN_PROGRESS", "current_step": "...", "context": {...} }
```

**GET** `/instances` **API-KEY** Lista instancias con filtros opcionales

**REQUEST BODY**

Query params: flow\_name, status, correlation\_id

**RESPONSE**

```
[{ "id": "uuid", "status": "...", "flow_name": "..." }]
```

**POST** `/instances/{id}/signal` **API-KEY** Envía señal a un human\_task en WAITING\_SIGNAL

**REQUEST BODY**

```
{ "step_name": "aprobacion_gerencia", "signal": "approve", "actor_id": "...", "signal_data": {...} }
```

**RESPONSE**

```
{ "instance_id": "uuid", "step_name": "...", "signal": "approve" }
```

## 9.3 Entities y relaciones

**POST****/entities/{type}****API-KEY**

Crea un nuevo entity

**REQUEST BODY**

```
{ "entity_id": "...", "campos": { "nombre": "...", "ci": "..." } }
```

**RESPONSE**

```
{ "entity_type": "nino", "entity_id": "...", "estado": "REGISTRADO" }
```

**POST****/entities/{type}/{id}/transition****API-KEY**

Ejecuta una transición de estado

**REQUEST BODY**

```
{ "via": "activar", "campos": {...} }
```

**RESPONSE**

```
{ "estado": "ACTIVO", "event": "nino.activado" }
```

**PATCH****/entities/{type}/{id}****API-KEY**

Actualiza campos (valida invariantes)

**REQUEST BODY**

```
{ "campos": { "nivel": "secundaria" } }
```

**RESPONSE**

```
{ "entity_id": "...", "campos": {...} }
```

**GET****/entities/{type}/{id}/360****API-KEY**

Vista 360 completa del entity

**RESPONSE**

```
{ estado, campos, relaciones_activas, procesos_activos, alertas, timeline }
```

**POST****/relations/{type}****API-KEY**

Crea una nueva relación

**REQUEST BODY**

```
{ "from_id": "...", "to_id": "...", "campos": {...} }
```

**RESPONSE**

```
{ "relation_type": "...", "from_id": "...", "to_id": "...", "estado": "PENDIENTE" }
```

**POST** `/relations/{type}/{id}/transition` **API-KEY** Transiciona el estado de la relación

**REQUEST BODY**

```
{ "via": "activar" }
```

**RESPONSE**

```
{ "estado": "ACTIVA", "event": "inscripcion.activada" }
```

## 9.4 Composer (IA)

**POST** `/compose` **API-KEY** Genera borrador .flow desde descripción en lenguaje natural

**REQUEST BODY**

```
{ "description": "Proceso de alta de servicio de agua para hogares...", "base_flow": "optional" }
```

**RESPONSE**

```
{ "draft_id": "uuid", "name": "...", "status": "pending" }
```

**GET** `/drafts` **API-KEY** Lista borradores pendientes de revisión

**RESPONSE**

```
[{ "id": "uuid", "name": "...", "status": "pending", "created_at": "..." }]
```

**POST** `/drafts/{id}/approve` **API-KEY** Aprueba el borrador y lo despliega como flow

**REQUEST BODY**

```
{ "version": "1.0.0", "description": "..." }
```

**RESPONSE**

```
{ "flow_id": "uuid", "status": "registered" }
```

**POST** `/drafts/{id}/reject` **API-KEY** Rechaza el borrador con comentario

**REQUEST BODY**

```
{ "comment": "Falta la etapa de validación de identidad" }
```

**RESPONSE**

```
{ "draft_id": "uuid", "status": "rejected" }
```

## 9.5 Observabilidad

**GET** **/health** Liveness probe — devuelve 200 si el proceso responde

**GET** **/ready** Readiness probe — verifica conectividad con DB y dependencias

**GET** **/metrics** Métricas en formato Prometheus (http\_requests\_total, dsl\_parse\_total, etc.)

## Despliegue y stack tecnológico

# 10 Despliegue y operación

## 10.1 Modelo de despliegue

### Desarrollo / Staging

```
# Primer arranque
cp .env.example .env
# Editar TELEFLOW_API_KEY, LLM_API_KEY, etc.

docker compose up --build -d

# Verificar servicios
docker compose ps
curl http://localhost:8000/health
```

Un solo servidor. Una imagen Docker para los 5 servicios Python. El comando selecciona el servicio via `uvicorn teleflow.<svc>.main:app`.

### Producción — Kubernetes / RKE2

```
# Deploy con Helm
helm upgrade --install teleflow \
./helm/teleflow \
--namespace teleflow \
--create-namespace \
-f values-production.yaml

# Escalar executor
kubectl scale deployment \
teleflow-executor-service \
--replicas=3 \
-n teleflow
```

Namespace dedicado. StatefulSets para Postgres, Redis y RabbitMQ. Deployments con 2-3 réplicas para los servicios sin estado.

## 10.2 Réplicas recomendadas en producción

| Servicio         | Tipo K8s    | Réplicas mínimas | Notas                                             |
|------------------|-------------|------------------|---------------------------------------------------|
| api-gateway      | Deployment  | 2                | Sin estado. Detrás de Ingress/LoadBalancer        |
| parser-service   | Deployment  | 2                | Sin estado. CPU-bound (Lark LALR)                 |
| registry-service | Deployment  | 2                | Sin estado. Lee de Postgres                       |
| executor-service | Deployment  | 3                | Semáforo interno por réplica (WORKER_CONCURRENCY) |
| composer-service | Deployment  | 1                | Llama a API externa. Sin estado local             |
| review-ui        | Deployment  | 2                | Nginx estático                                    |
| postgres         | StatefulSet | 1+               | PVC dedicado. Recomendado: HA con patroni         |
| redis            | StatefulSet | 1                | Sentinel para HA en prod                          |
| rabbitmq         | StatefulSet | 3                | Cluster quorum queue recomendado                  |

## 10.3 Variables de entorno

| Variable            | Requeri-da | Default           | Descripción                                          |
|---------------------|------------|-------------------|------------------------------------------------------|
| TELEFLOW_API_KEY    | Sí         | dev-key-change-me | API key para el gateway. Cambiar en producción       |
| DATABASE_URL        | Sí         | —                 | postgresql+asyncpg://user:pass@host:5432/teleflow    |
| REDIS_URL           | Sí         | —                 | redis://host:6379/0                                  |
| RABBITMQ_URL        | Sí         | —                 | amqp://user:pass@host:5672/                          |
| POSTGRES_PASSWORD   | Sí         | teleflow          | Password del usuario Postgres                        |
| RABBITMQ_PASSWORD   | Sí         | teleflow          | Password del usuario RabbitMQ                        |
| LLM_PROVIDER        | No         | stub              | anthropic   openai   ollama   stub                   |
| LLM_API_KEY         | No         | —                 | Clave del proveedor LLM (requerida si no stub)       |
| LLM_MODEL           | No         | —                 | Modelo a usar (ej: claude-fable-5, gpt-4o)           |
| LLM_BASE_URL        | No         | —                 | Para Ollama: http://host:11434                       |
| WORKER_CONCURRENCY  | No         | 10                | Semáforo del executor: instancias concurrentes       |
| TIMER_SCAN_INTERVAL | No         | 60                | Segundos entre escaneos del rule engine on_timer     |
| RATE_LIMIT_RPM      | No         | 120               | Requests por minuto por API key en el gateway        |
| GRAFANA_PORT        | No         | 3001              | Puerto del host para Grafana (default 3001, no 3000) |
| GRAFANA_PASSWORD    | No         | admin             | Password del admin de Grafana                        |

### ⚠️ Notas de entorno Windows / Docker Desktop

- Las rutas con espacios («Mi unidad», «Servicios SAS») no funcionan como bind mounts en Docker Desktop para Windows. Prometheus y Grafana usan Dockerfiles propios que copian la config con COPY (en lugar de bind mount).
- El puerto 3000 puede estar ocupado por wsrelay. Usar `GRAFANA_PORT=3001`.
- Para enviar JSON con acentos desde PowerShell, usar `[Text.Encoding]::UTF8.GetBytes(json)` como body del request.

## 10.4 CLI tflow — comandos disponibles

| Comando                       | Descripción                                 | Ejemplo                                    |
|-------------------------------|---------------------------------------------|--------------------------------------------|
| <code>`tflow validate`</code> | Valida un .tflow localmente sin desplegarlo | <code>`tflow validate ceibal.tflow`</code> |

|                                |                                         |                                                                             |
|--------------------------------|-----------------------------------------|-----------------------------------------------------------------------------|
| <code>`tf flow deploy`</code>  | Parsea y registra en el registry        | <code>`tf flow deploy ceibal.tflow --name ceibal --version 1.0.0`</code>    |
| <code>`tf flow flows`</code>   | Lista flows registrados                 | <code>`tf flow flows`</code>                                                |
| <code>`tf flow execute`</code> | Dispara una instancia de proceso        | <code>`tf flow execute --flow ceibal --payload '{...}'`</code>              |
| <code>`tf flow status`</code>  | Muestra estado de una instancia         | <code>`tf flow status &lt;instance_id&gt;`</code>                           |
| <code>`tf flow signal`</code>  | Envía señal a un human_task             | <code>`tf flow signal &lt;id&gt; --step aprobacion --signal approve`</code> |
| <code>`tf flow compose`</code> | Genera borrador .tflow con IA           | <code>`tf flow compose --desc 'proceso de alta de cliente'`</code>          |
| <code>`tf flow drafts`</code>  | Lista borradores pendientes de revisión | <code>`tf flow drafts`</code>                                               |
| <code>`tf flow approve`</code> | Aprueba y despliega un borrador         | <code>`tf flow approve &lt;draft_id&gt; --version 1.0.0`</code>             |

## 10.5 Schema de base de datos

Gestionado con Alembic. La migración inicial está en `alembic/versions/0001_initial.py`. No se usa `create_all()` en producción.

**flow\_definitions** UNIQUE(name, version). Una vez registrada, la row es inmutable.

|                         |                                          |
|-------------------------|------------------------------------------|
| <code>id</code>         | UUID PK                                  |
| <code>name</code>       | varchar(200)                             |
| <code>version</code>    | varchar(50)                              |
| <code>source</code>     | text                                     |
| <code>ast</code>        | JSONB                                    |
| <code>checksum</code>   | varchar(64) — SHA-256 del source         |
| <code>status</code>     | varchar(30) — registered/active/archived |
| <code>created_at</code> | timestampz                               |

**process\_instances** context\_cursor es el estado durable del durable sleep.

|                                      |                                                                     |
|--------------------------------------|---------------------------------------------------------------------|
| <code>id</code>                      | UUID PK                                                             |
| <code>flow_name</code>               | varchar                                                             |
| <code>flow_version</code>            | varchar                                                             |
| <code>correlation_id</code>          | varchar — índice                                                    |
| <code>status</code>                  | varchar(30) — TRIGGERED/IN_PROGRESS/WAITING_SIGNAL/COMPLETED/FAILED |
| <code>context</code>                 | JSONB — incluye _cursor: {stage, step}                              |
| <code>current_step</code>            | varchar                                                             |
| <code>error</code>                   | JSONB nullable                                                      |
| <code>created_at / updated_at</code> | timestampz                                                          |

**signals** signal\_key UNIQUE garantiza exactamente-una-ejecución.

|                          |                                          |
|--------------------------|------------------------------------------|
| <code>id</code>          | UUID PK                                  |
| <code>instance_id</code> | UUID FK → <code>process_instances</code> |
| <code>step_name</code>   | varchar                                  |
| <code>signal</code>      | varchar(100)                             |
| <code>actor_id</code>    | varchar nullable                         |
| <code>signal_data</code> | JSONB                                    |
| <code>signal_key</code>  | varchar(200) UNIQUE — idempotencia       |
| <code>processed</code>   | boolean                                  |
| <code>created_at</code>  | timestamptz                              |

**entity\_state** UNIQUE(entity\_type, entity\_id). Upsert en transiciones.

|                          |              |
|--------------------------|--------------|
| <code>id</code>          | UUID PK      |
| <code>entity_type</code> | varchar(100) |
| <code>entity_id</code>   | varchar(100) |
| <code>estado</code>      | varchar(50)  |
| <code>campos</code>      | JSONB        |
| <code>updated_at</code>  | timestamptz  |

**entity\_events** Log inmutable de eventos. Fuente para la Vista 360 y `on_timer`.

|                          |                      |
|--------------------------|----------------------|
| <code>id</code>          | UUID PK              |
| <code>entity_type</code> | varchar(100)         |
| <code>entity_id</code>   | varchar(100)         |
| <code>event_name</code>  | varchar(200)         |
| <code>payload</code>     | JSONB                |
| <code>occurred_at</code> | timestamptz — índice |

**flow\_drafts** Los borradores aprobados se despliegan como `flow_definition`.

|                          |                                         |
|--------------------------|-----------------------------------------|
| <code>id</code>          | UUID PK                                 |
| <code>name</code>        | varchar                                 |
| <code>description</code> | text                                    |
| <code>source</code>      | text — borrador generado                |
| <code>base_source</code> | text nullable — original si es edición  |
| <code>status</code>      | varchar(30) — pending/approved/rejected |
| <code>comments</code>    | JSONB array                             |

**rule\_timer\_log** UNIQUE(rule\_name, subject\_key). Evita doble disparo de `on_timer`.

|                          |              |
|--------------------------|--------------|
| <code>id</code>          | UUID PK      |
| <code>rule_name</code>   | varchar(200) |
| <code>subject_key</code> | varchar(300) |
| <code>fired_at</code>    | timestamptz  |

## 11 Stack tecnológico



| Componente     | Tecnología                   | Justificación                                                                                                   |
|----------------|------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Framework web  | FastAPI + uvicorn            | Async nativo, tipado con Pydantic v2, OpenAPI automático, performance comparable a Go para I/O bound            |
| Parser DSL     | Lark (LALR)                  | Gramática EBNF en Python. Performance LALR determinístico. Transformer → dataclasses. Ver ADR-001               |
| DAG engine     | networkx                     | Construcción y traversal del DAG del flow. Detección de ciclos. API intuitiva para grafos dirigidos             |
| HTTP client    | httpx                        | Async nativo, HTTP/2, timeouts configurables. Usado en adapters REST y en el gateway                            |
| AMQP client    | aio-pika                     | RabbitMQ async nativo asyncio. Topic exchange para routing por evento                                           |
| ORM            | SQLAlchemy 2 async + asyncpg | Async I/O nativo hacia Postgres. Mapped columns con Python typing. Sin ORMs que bloqueen el event loop          |
| Migraciones    | Alembic                      | Versionado de schema. No create_all() en producción. Historia completa de cambios                               |
| Logging        | structlog                    | Logging estructurado JSON. instance_id, flow_name, step_name en cada línea                                      |
| Type checking  | mypy --strict                | Sin Any sin justificar. Pydantic v2 para modelos API, dataclasses para AST interno                              |
| Testing        | pytest-asyncio               | Sin unittest. Fixtures async para todos los tests. 26/26 tests pasan                                            |
| Frontend       | React + Vite                 | UI liviana para PR-style review de .tflow. Diff viewer LCS personalizado                                        |
| Observabilidad | Prometheus + Grafana         | Dashboards técnicos (latencia, error rate) y de negocio (procesos completados, backlog)                         |
| Contenedores   | Docker + Helm/RKE2           | Una imagen para 5 servicios Python. Helm charts para Kubernetes. docker compose para dev                        |
| Redis          | redis.asyncio (pub/sub)      | Durable sleep reactivation. Cache de ASTs con TTL 30s. get_message en lugar de listen() para networking estable |
| Mensajería     | RabbitMQ 3.13                | Topic exchange. aio-pika consumer en executor. management UI en :15672. Ver ADR-004                             |

## 11.1 Convenciones de código

### ✓ Hacer

- mypy --strict en todos los servicios
- Pydantic v2 para request/response models
- Dataclasses para nodos AST (no Pydantic)
- structlog con campos contextuales siempre
- pytest-asyncio para todos los tests
- Alembic para cualquier cambio de schema
- /health (liveness) y /ready (readiness) en cada svc
- /metrics en formato Prometheus en cada svc

### ✗ Nunca hacer

- create\_all() o Base.metadata.create\_all() en prod
- Any sin justificar en mypy
- unittest (usar pytest siempre)
- Exponer servicios internos fuera de la red Docker
- Credenciales hardcodeadas (usar \${env.VAR} en .tflow)
- Bloquear el event loop con operaciones síncronas pesadas
- Modificar una versión ya registrada en el registry

## 12 Roadmap de implementación

### Fase 1 — Motor básico

**COMPLETADO**

- ✓ parser-service (Lark + AST + validación)
- ✓ registry-service (CRUD Postgres + semver)
- ✓ executor-service (sequential stages + REST adapters)
- ✓ api-gateway (auth + rate limit + routing)

### Fase 2 — Dominio + long-running

**COMPLETADO**

- ✓ entity + relation + rule en parser y executor
- ✓ executor: parallel stages + durable sleep + signals
- ✓ Adapters AMQP + SMTP + SMS
- ✓ Prometheus + Grafana + dashboards

### Fase 3 — IA + 360

**COMPLETADO**

- ✓ composer-service (abstracción LLM: Claude/GPT-4o/Ollama/Stub)
- ✓ review-ui (React PR-style + LCS diff viewer)
- ✓ view360 endpoint con 5 fuentes en paralelo
- ✓ Templates TMForum (6 procesos tipo)

### Fase 4 — Producción

**EN PROGRESO**

- ✓ Helm charts K8s/RKE2 con values por entorno
- ✓ CLI tflow (validate/deploy/execute/signal/compose/approve)
- ✓ Plugin VS Code (Tree-sitter + syntax highlighting)
- ✓ SLA + runbooks + disaster recovery

## Decisiones de arquitectura (ADRs)

# 11 Decisiones de arquitectura (ADRs)

Las ADRs documentan las decisiones técnicas significativas tomadas durante el diseño de TeleFlow. Cada ADR captura el contexto, las alternativas y las consecuencias para que el equipo pueda entender y evaluar los trade-offs en el futuro.

### ADR-001 Lark + LALR para el parser DSL

ACCEPTED

#### DECISIÓN

Usar Lark con el algoritmo LALR como parser del lenguaje .tflow. La gramática EBNF vive en `teleflow/dsl/teleflow.lark` y es la única fuente de verdad del lenguaje.

#### ALTERNATIVAS CONSIDERADAS

- PLY (lex/yacc en Python) — más verboso
- Parsimonious (PEG) — backtracking sin límite
- Tree-sitter — excelente para IDE, difícil integrar en Python runtime
- Parser combinador manual — frágil a largo plazo

#### RAZONAMIENTO

Lark ofrece la mejor relación entre legibilidad de gramática (EBNF nativo en Python), performance LALR determinístico en producción, y extensibilidad. El transformer Visitor/Transformer de Lark convierte el parse tree en dataclasses tipados con código mínimo. Tree-sitter se reserva para el plugin de VS Code en Fase 4.

#### CONSECUENCIAS

La gramática EBNF es la fuente de verdad: cualquier extensión del DSL requiere modificar `teleflow.lark` primero. El transformador debe actualizarse para cada nueva regla gramatical. Los errores de sintaxis incluyen línea/columna gracias al error reporting de Lark.

### ADR-002 Durable sleep vía Redis pub/sub + Postgres

ACCEPTED

#### DECISIÓN

Las instancias en `WAITING_SIGNAL` persisten su contexto completo en Postgres. Redis pub/sub las reactiva en milisegundos. La señal es idempotente por `signal_key` único (UNIQUE constraint).

#### ALTERNATIVAS CONSIDERADAS

- Polling activo — consume recursos, latencia variable
- Timer externo (cron job) — dependencia nueva, complejidad operativa
- Temporal.io — excelente pero requiere infraes-

#### RAZONAMIENTO

El estado durable en Postgres garantiza que el executor puede reiniciarse sin perder instancias dormidas. Redis pub/sub provee reactivación con latencia de milisegundos sin polling activo. El modelo es conceptualmente idéntico al de Temporal.io pero con componentes estándar ya presentes en el stack. La idempotencia de señales (`signal_key` UNIQUE) garantiza exactamente-una-ejecución incluso con reintentos del cliente.

#### CONSECUENCIAS

estructura propia y licencia

- Celery + Redis — acoplamiento fuerte, no soporta estado durable nativo

El executor debe suscribirse a Redis al arrancar y recuperar instancias dormidas desde Postgres (`_recover()`). En entornos con networking inestable (Docker Desktop) el listener usa `get_message` con polling suave en lugar de `listen()` bloqueante.

## ADR-003 LLM configurable por instancia de organismo

**ACCEPTED**

### DECISIÓN

El proveedor LLM se configura via variables de entorno (`LLM_PROVIDER`, `LLM_API_KEY`, `LLM_MODEL`, `LLM_BASE_URL`). La interfaz `LLMProvider` abstrae `anthropic/openai/ollama/stub`.

### ALTERNATIVAS CONSIDERADAS

- LLM fijo (solo Anthropic/Claude) — no sirve para organismos con restricciones
- Configuración en DB — complejidad operativa sin beneficio
- Plugin system — sobre-ingeniería para 4 proveedores

### RAZONAMIENTO

Cada organismo tiene restricciones propias: OSE puede usar Claude API; Antel puede requerir modelo self-hosted por regulación; el Ministerio puede necesitar datos en territorio nacional. La interfaz `LLMProvider` es un ABC de Python con un único método `generate(prompt) → str`. El cambio de proveedor es una variable de entorno, no un deploy de código.

### CONSECUENCIAS

`composer-service` implementa `AnthropicProvider`, `OpenAIProvider`, `OllamaProvider` y `StubProvider`. `StubProvider` genera un esqueleto sintácticamente válido offline, útil para desarrollo y testing. Si `LLM_API_KEY` no está configurada, el servicio cae automáticamente a `Stub`.

## ADR-004 RabbitMQ incluido en el stack

**ACCEPTED**

### DECISIÓN

RabbitMQ se incluye como servicio propio en `docker-compose` y los Helm charts. No se depende del broker del organismo cliente para la instalación inicial.

### ALTERNATIVAS CONSIDERADAS

- Kafka — sobre-dimensionado para el volumen de eventos de un organismo
- Broker del organismo (RabbitMQ/Kafka externo) — bloquea el onboarding inicial
- Sin broker (directo via Postgres NOTIFY) — limita la integración con sistemas externos

### RAZONAMIENTO

Simplifica el onboarding: `docker compose up` funciona sin configuración externa. Los organismos que ya tienen broker propio pueden configurar TeleFlow para usarlo desactivando el RabbitMQ del Compose (variable `RABBITMQ_URL` apuntando al externo). La migración a broker externo es un cambio de configuración, no de código.

### CONSECUENCIAS

El stack es autosuficiente desde el primer día. El executor usa `aio-pika` para AMQP async. Los exchanges son de tipo `topic` con routing key = nombre del evento (`nino.activado`, `inscripcion.activada`, etc.).

**ADR-005 Aislamiento por instancia — no multi-tenant****ACCEPTED****DECISIÓN**

Cada organismo tiene su propia instancia de TeleFlow con su propia base de datos. No hay row-level security ni schema switching.

**ALTERNATIVAS CONSIDERADAS**

- Multi-tenant con RLS en Postgres — viable pero suma complejidad operativa y de seguridad
- Multi-tenant con schemas separados — facilita backups por tenant pero complica las migraciones Alembic
- SaaS compartido — contradice el requisito de datos aislados de organismos públicos

**RAZONAMIENTO**

Modelo más simple de operar para organismos con equipos IT propios. Cada organismo conoce exactamente qué datos tiene y dónde. El aislamiento es físico, no lógico: imposible data leak entre organismos por bug de aplicación. Los templates y procesos se comparten vía Git, no vía DB compartida.

**CONSECUENCIAS**

Las actualizaciones de la plataforma se aplican via git pull + docker compose pull + docker compose up --build en cada instancia. No hay mecanismo automático de propagación. Los organismos grandes pueden operar múltiples instancias (ej: una por línea de negocio) con configuración independiente.

## 11.1 Resumen de decisiones

| ADR | Título                    | Estado   | Impacto principal                                                                   |
|-----|---------------------------|----------|-------------------------------------------------------------------------------------|
| 001 | Lark + LALR               | ACCEPTED | Gramática EBNF como fuente de verdad. Extensiones del DSL empiezan en teleflow.lark |
| 002 | Durable sleep             | ACCEPTED | Postgres + Redis. Recovery automático en restart. Señales idempotentes              |
| 003 | LLM configurable          | ACCEPTED | anthropic / openai / ollama / stub via env vars. Sin recompilación                  |
| 004 | RabbitMQ en el stack      | ACCEPTED | Stack autosuficiente desde docker compose up. Migración a broker externo = env var  |
| 005 | Aislamiento por instancia | ACCEPTED | Una DB por organismo. Sin multi-tenancy. Aislamiento físico                         |

## Apéndice A — Dominio Ceibal (ceibal.tflow)

text(size: 9pt)[ El archivo `examples/ceibal.tflow` es el ejemplo canónico del DSL. Incluye los 9 tipos de bloque y cubre el flujo completo del dominio educativo de Ceibal. ]

```
tflow · examples/ceibal.tflow - entity nino
```

```
entity «nino» { description: «Estudiante registrado en Ceibal» fields { ci: string required unique nombre: string required
fecha_nac: date required departamento: string nivel: enum[«primaria», «secundaria», «utu»] dispositivo: string optional
tutor_email: string optional } lifecycle { initial: «REGISTRADO» states [«REGISTRADO», «ACTIVO», «INSCRIPTO»,
«GRADUADO», «INACTIVO»] transitions { REGISTRADO -> ACTIVO via «activar» ACTIVO -> INSCRIPTO via
«inscribir» INSCRIPTO -> ACTIVO via «desinscribir» ACTIVO -> GRADUADO via «graduar» ACTIVO -> INACTIVO
via «desactivar» INACTIVO -> ACTIVO via «reactivar» } } events { on_transition «activar» emit «nino.activado»
on_transition «inscribir» emit «nino.inscripto» on_transition «graduar» emit «nino.graduado» on_field_change «nivel»
emit «nino.nivel_cambiado» } invariants { «edad >= 5 AND edad <= 18» «nivel != null WHEN estado == INSCRIPTO» } }
```

```
tflow · examples/ceibal.tflow - reglas event-driven
```

```
rule «activar_acceso_al_inscribirse» { on_event: «inscripcion.activada» execute: process.activacion_acceso_plataforma
with { nino_id: event.from_id curso_id: event.to_id } }

rule «detectar_abandono» { on_timer { after: 30 days since: «inscripcion.activada» condition:
days_since(relation.inscripcion.ultimo_acceso) >= 30 OR relation.inscripcion.ultimo_acceso == null } execute:
process.reenganche_estudiante with { nino_id: relation.from_id curso_id: relation.to_id } }
```

## Apéndice B — Quick start

### B.1 Instalar y arrancar

```
# 1. Clonar y configurar
git clone <repo>
cd govtech-platform
cp .env.example .env

# 2. Levantar el stack completo
docker compose up --build -d

# 3. Verificar servicios
docker compose ps
curl -s http://localhost:8000/health

# 4. Desplegar con la CLI
pip install -e .
export TELEFLOW_URL=http://localhost:8000
export TELEFLOW_API_KEY=dev-key-change-me
tflow validate examples/ceibal.tflow
tflow deploy examples/ceibal.tflow --name ceibal --version 1.0.0
```

### B.2 Ciclo de vida completo — Durable sleep

```
# Desplegar
tflow deploy examples/venta_internet_hogar.tflow \
  --name venta_internet_hogar --version 1.0.0

# Ejecutar proceso
tflow execute --flow venta_internet_hogar \
  --payload '{"cliente_id":"cli-001","producto":"internet_100"}'

# Verificar estado (duerme en aprobacion_gerencia)
tflow status <instance_id>
# status: WAITING_SIGNAL, current_step: aprobacion_gerencia

# Enviar señal de aprobación
tflow signal <instance_id> \
  --step aprobacion_gerencia --signal approve --actor juan.perez

# Verificar completado
tflow status <instance_id>
# status: COMPLETED
```

### B.3 Observabilidad

```
# Prometheus — scraping de 5 targets
open http://localhost:9090/targets

# Grafana — dashboards tecnicos y de negocio
open http://localhost:3001
# admin / admin (cambiar en produccion)

# Logs estructurados del executor
docker compose logs -f executor-service
```

---

TeleFlow Platform · Documento de Arquitectura v1.0 · Junio 2026 · BPFocus · COPP-IA

*Confidencial - No distribuir sin autorizacion*